

Configuration

QF-AgentOS — An honest benchmark and evidence harness for quantum finance

QF-AgentOS contributors

Version 0.1.0 — 2026-07-16

Contents

Observability	1
Determinism & numerics	1
Simulation budgets	1
Governance / registry	2
REST API	2
Credentials (secrets)	2
Spec-level policy (per run, not env)	3
Extras	3

All settings are environment variables with the **QF_** prefix, read once per process (see [.env.example](#)). Credentials are held as `SecretStr` — never logged, never printed in a repr, never written into an evidence bundle.

```
export QF_LOG_FORMAT=json
export QF_REGISTRY_BACKEND=mlflow
qf-agent solve spec.yaml
```

Observability

Variable	Default	Meaning
QF_LOG_LEVEL	INFO	DEBUG / INFO / WARNING / ERROR / CRITICAL
QF_LOG_FORMAT	text	text or json (run-id-correlated structured logs)
QF_TRACING_ENABLED	false	OpenTelemetry spans per agent step (needs the <code>otel</code> extra)

Determinism & numerics

Variable	Default	Meaning
QF_DEFAULT_SEED	7	Default seed (a spec's <code>execution_policy.seed</code> overrides)
QF_NUMERIC_REL_TOL	1e-6	Relative tolerance for objective comparisons

Same spec + same seed $\Rightarrow$ byte-identical `evidence_digest`.

Simulation budgets

Guard-rails against runaway resource use. The planner **abstains** (honestly) rather than exceeding them.

Variable	Default	Meaning
QF_STATEVECTOR_QUBIT_LIMIT	22	Max qubits for exact statevector QAOA
QF_BRUTEFORCE_QUBIT_LIMIT	20	Max qubits for brute-force QUBO ground truth

Governance / registry

Variable	Default	Meaning
QF_EVIDENCE_DIR	evidence	File-store root for evidence bundles
QF_REGISTRY_BACKEND	file	file · mlflow · postgres
QF_MLFLOW_TRACKING_URI	—	MLflow tracking URI (default ./mlruns); needs the mlflow extra
QF_MLFLOW_EXPERIMENT	qf-agentos	MLflow experiment name
QF_POSTGRES_DSN	—	e.g. postgresql+psycopg://user:pass@host/db; needs the postgres extra

```
# MLflow
export QF_REGISTRY_BACKEND=mlflow QF_MLFLOW_TRACKING_URI=http://mlflow:5000
# Postgres
export QF_REGISTRY_BACKEND=postgres QF_POSTGRES_DSN='postgresql+psycopg://qf:pw@db/qf'
```

QF_REGISTRY_BACKEND=postgres without a DSN fails fast with a clear error.

REST API

Variable	Default	Meaning
QF_API_KEYS	"" (open)	Comma-separated accepted X-API-Key values
QF_API_RATE_LIMIT_PER_MINUTE	60	Per-client limit on the solve endpoints
QF_API_MAX_INVENTORY	2000	Max problem size over the API (all families) → 413
QF_API_JOB_WORKERS	2	Concurrent async solve workers
QF_API_MAX_JOBS	256	Retained job records (LRU)

With QF_API_KEYS unset the API is **open** and logs a startup warning. Always set it in any shared deployment.

Credentials (secrets)

Never commit these or paste them anywhere. Export them in your own shell.

Variable	Default	Meaning
QF_IBM_TOKEN	—	IBM Quantum API key → enables qaoa_ibm
QF_IBM_CHANNEL	ibm_quantum_platform	IBM channel; or ibm_cloud. (<i>The legacy ibm_quantum channel was retired.</i>)
QF_IBM_INSTANCE	—	Instance CRN / hub-group-project (optional; auto-selected on the Open plan)
QF_IBM_BACKEND	—	Pin a device; otherwise least-busy is chosen
QF_DWAVE_TOKEN	—	D-Wave Leap token → enables dwave_hybrid

Credentials alone are **not** sufficient to run a QPU. Real hardware also requires qpu_backend: ibm|dwave in the spec, autonomy_level: L3, a sufficient max_qpu_budget_usd, and explicit --approve. See [real-qpu.md](#).

Spec-level policy (per run, not env)

Set in the spec's `execution_policy` — these travel with the problem and into the evidence bundle:

```
execution_policy:
  compare_classical: true
  allow_gate_model: true
  allow_quantum_annealing: true
  qpu_backend: sim          # sim | ibm | dwave
  autonomy_level: L2        # L0...L4
  max_qpu_budget_usd: 0
  max_effective_qubits: 20
  qaoa_reps: 1
  shots: 4096
  seed: 7
  noisy_simulation: false   # + noise_two_qubit_error, readout_error
```

Extras

Extra	Installs	Enables
(none)	scipy, pydantic	Classical MILP/LP, the full pipeline
qiskit	qiskit, qiskit-aer	qaoa_sim, noisy sim, QAE, tensor-network analysis
ibm	+ qiskit-ibm-runtime	qaoa_ibm (real hardware)
dwave	dwave-ocean-sdk	dwave_hybrid (real annealer)
pennylane	pennylane	qaoa_pennylane
server	fastapi, uvicorn	REST API + Studio
mlflow / postgres	mlflow / sqlalchemy+psycpg	Registries
otel	opentelemetry	Tracing
all / dev	everything / + test tooling	